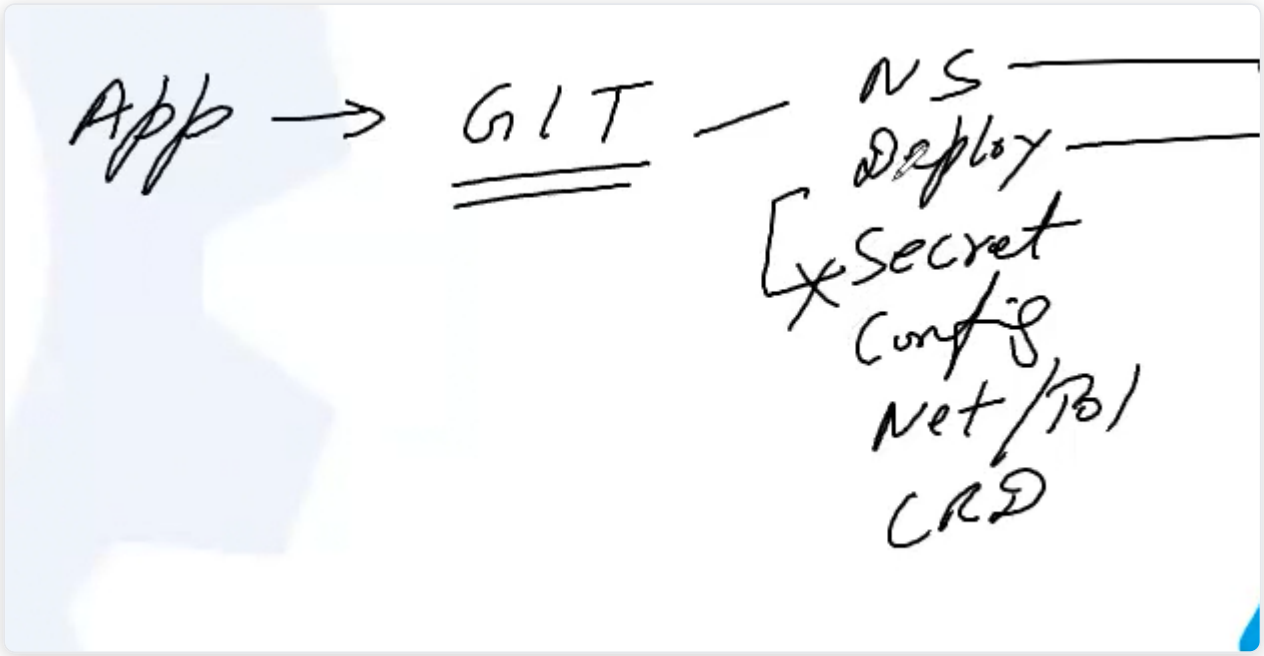
ARGO CD GITOPS › MY LEARNING NOTES › SESSION 05

Screenshots: Sync Waves

Raw screenshots captured while watching this session (wave ordering, common mistakes, health check overrides via argocd-cm), kept alongside the written notes (05-sync-waves.md).

27 screenshots · captured 2026-09-13, 12:34 PM - 3:09 PM


SYNC WAVES
ARGO CD + KUBERNETES


App → GIT
NS
Deploy
Secret
Config
Net/BI
CRD

Imagine deploying a Kubernetes application where the ConfigMap isn't ready before the Deployment starts. The pods crash because they can't find their configuration.

Or think about a Service that comes up before the Deployment exists, leaving you with errors until the workload is available. Your pods cannot run if the namespace does not exist.

These dependency issues are frustrating and often happen when Argo CD tries to apply everything at once.

That's where sync waves come in.

Sync waves let you control the order of deployments so that dependent resources are created at the right time. Instead of hoping resources line up correctly, you define the sequence and avoid unnecessary failures.

Sync waves define deployment sequences. Argo CD deploys resources in numerical order based on their wave annotations. Without sync waves, Argo CD deploys all resources simultaneously. This creates issues when resources depend on each other. Sync waves give you fine-grained control over deployment ordering.

Argo CD Sync Waves

Sync waves let you group resources and deploy them in sequence. Resources in the same wave deploy together. Lower numbered waves deploy before higher ones.



ArgoCD processes waves from lowest to highest number. Negative numbers run before zero. Resources without a wave annotation default to wave 0.

Argo CD Sync Waves

Resources are deployed in numerical order based on their wave value (from lowest to highest).

- Default Wave: If you don't specify a wave, the resource is automatically assigned to Wave 0.
- Negative Waves: Waves can have negative values, allowing you to deploy foundational components (like Namespaces, RBAC rules, or custom resource definitions) before anything else.
- Positive Waves: Waves can also be positive numbers, which is useful for things that depend on the core application being fully operational first.
- Ordering Fallbacks: Within the same wave number, resources are sorted and applied by kind (e.g., Namespaces first, then Services, then Deployments) and then alphabetically by name.

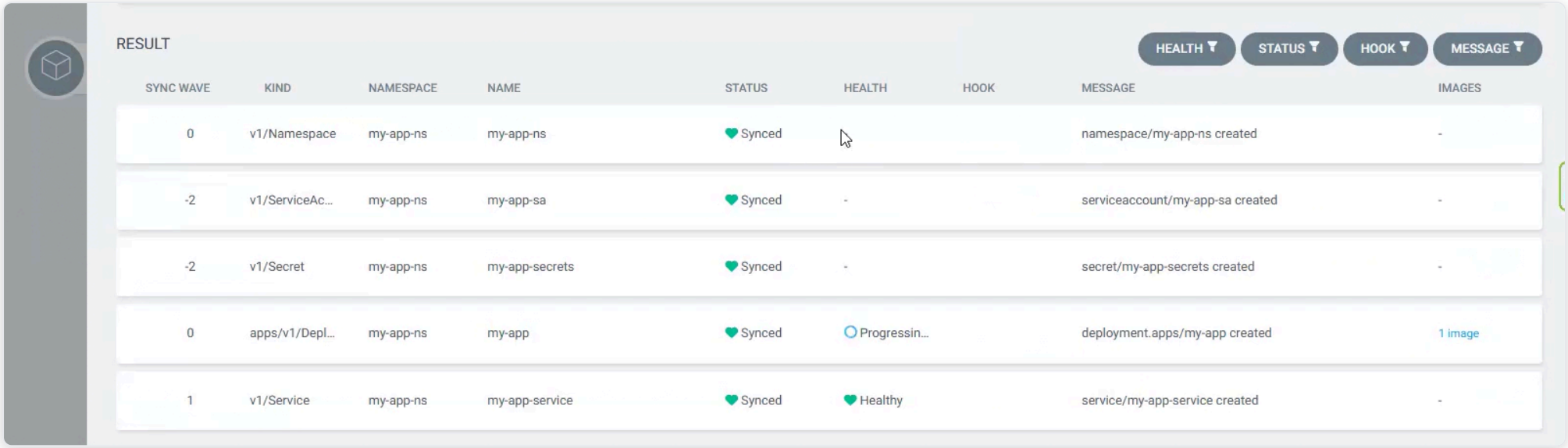
Argo CD Sync Waves

Wave Numbering Strategy

- Wave -5 to -1: Infrastructure (namespaces, CRDs)
- Wave 0 to 2: Configuration (configs, secrets)
- Wave 3 to 5: Applications (deployments, services)
- Wave 6+: External resources (ingress, monitoring)

Code | Blame | 72 lines (71 loc) · 1.39 KB

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: my-app-ns
  labels:
     area: my-app-ns
  annotations:
    argocd.argoproj.io/sync-wave: "-5"
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-app-sa
  namespace: my-app-ns
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
---
apiVersion: v1
kind: Secret
metadata:
  name: my-app-secrets
  namespace: my-app-ns
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
```


RESULT
HEALTH
STATUS
HOOK
MESSAGE
SYNC WAVE
KIND
NAMESPACE
NAME
STATUS
HEALTH
HOOK
MESSAGE
IMAGES
0
v1/Namespace
my-app-ns
my-app-ns
Synced
namespace/my-app-ns created
-
-2
v1/ServiceAc...
my-app-ns
my-app-sa
Synced
-
serviceaccount/my-app-sa created
-
-2
v1/Secret
my-app-ns
my-app-secrets
Synced
-
secret/my-app-secrets created
-
0
apps/v1/Depl...
my-app-ns
my-app
Synced
Progressin...
deployment.apps/my-app created
1 image
1
v1/Service
my-app-ns
my-app-service
Synced
Healthy
service/my-app-service created
-

Wave	Purpose
-5	CRDs, Operators
-3	Namespaces
-1	RBAC, ServiceAccounts
0	ConfigMaps, Secrets
1	Databases, Caches
2	Migrations, Init Jobs
3	Applications
5	Ingress, External Services

```
# Show the current operation details, including resources being synced
argocd app get networknuts-app --show-operation

# List resources in the application
argocd app resources networknuts-app --output tree=detailed
```

Argo CD Sync Waves - Common Mistakes

Too many waves. Teams that add sync waves for every resource end up with ten-wave configurations where waves 3, 4, and 5 contain resources with no actual dependencies between them. Each wave adds latency to the sync because Argo CD waits for health before proceeding. Keep the number of waves proportional to the number of genuine dependencies.

Missing or incorrect readiness probes. Sync waves depend on Argo CD's health assessment. A Deployment is healthy when all desired replicas are ready. A Pod is healthy when all containers are running. If your readiness probe doesn't accurately reflect application readiness, the next wave starts before the current wave's resources are actually ready. The probe is the contract between your application and the sync wave system.

Resources stuck in unhealthy states blocking everything. If a resource in wave 1 enters an error state and never becomes healthy, all subsequent waves are blocked indefinitely. The sync stays in progress. You either fix the underlying issue, or use argocd app terminate-op to stop the sync and investigate.

Argo CD Sync Waves vs Kubernetes Probes

Kubernetes probes check if a container is running and ready.
Argo CD sync waves control the order in which resources are deployed.

Together, they ensure that a sync wave only finishes once the deployed resources pass their health checks.

How They Work Together

- Sync Waves (Ordering): Argo CD uses sync waves (via the argocd.argoproj.io/sync-wave annotation) to decide when to apply a manifest. Wave 0 deploys before wave 1, and Argo CD will not move to wave 1 until all resources in wave 0 reach a Healthy status.

- Probes (Health Signals): Argo CD evaluates the "Healthy" status of a Kubernetes resource (like a Deployment or StatefulSet) by looking at its underlying pods. Readiness probes tell Kubernetes—and by extension, Argo CD—when a pod is fully initialized and ready to accept traffic.

The first step in debugging is confirming that ArgoCD actually sees your sync wave annotations. A “typo” in the annotation key means ArgoCD treats the resource as wave 0.

Wrong annotation key:

```
# WRONG - missing the full annotation key
metadata:
  annotations:
    sync-wave: "1"  # ArgoCD ignores this

# CORRECT
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "1"
```

Argo CD sync wave gets stuck because Argo CD syncs resources wave by wave and refuses to move to the next wave until every resource in the current wave becomes fully synced and healthy.

Common causes of stuck waves:

Deployment with insufficient resources. The pod cannot be scheduled because the cluster does not have enough CPU or memory. The Deployment stays in "Progressing" state.

ImagePullBackOff. The container image does not exist or credentials are wrong. The Deployment never becomes healthy.

Readiness probe failure. The pod starts but the readiness probe never passes. Kubernetes reports the pod as not ready, and ArgoCD reports the Deployment as Progressing.

CRD not becoming Established. If a CRD has schema validation errors, it might be created but never reach the Established condition. The wave with the CRD stays stuck.

Fix a Stuck Sync Wave:

#1: Fix the stuck resource. This is the right answer in most cases. Figure out why the resource is unhealthy and fix it.

#2: Remove the stuck resource from the application. If the resource is optional, remove it from the Git repository and sync again. ArgoCD will prune it and the wave can complete.

#3: Override the health check for that resource type. You can configure ArgoCD to treat certain resource types as always healthy with a custom health check in the ArgoCD ConfigMap.

```
alok@gateway:~/argocd$ kubectl get ns
NAME                 STATUS   AGE
argocd               Active   14d
default              Active   21d
dev                  Active   7d
ingress-nginx        Active   5h15m
kube-node-lease      Active   21d
kube-public          Active   21d
kube-system          Active   21d
metallb-system       Active   5h14m
my-app-ns            Active   24m
myapp-production     Active   16m
prod                 Active   3d23h
production           Active   6d
alok@gateway:~/argocd$
alok@gateway:~/argocd$
alok@gateway:~/argocd$ kubectl -n argocd get cm
NAME                        DATA   AGE
argocd-cm                   10     14d
argocd-cmd-params-cm        0      14d
argocd-gpg-keys-cm          0      14d
argocd-notifications-cm     0      14d
argocd-rbac-cm              0      14d
argocd-ssh-known-hosts-cm   1      14d
argocd-tls-certs-cm         0      14d
kube-root-ca.crt            1      14d
alok@gateway:~/argocd$ kubectl -n argocd edit cm argocd-c
```

```
Step #1

kubectl edit configmap argocd-cm -n argocd

---
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-cm
  namespace: argocd
data:
  resource.customizations: |
    # Replace 'apps/CustomResource' or 'apiextensions.k8s.io/CustomResource' with your group/kind
    your-api-group/YourKind:
      health.lua: |
        hs = {}
        hs.status = "Healthy"
        hs.message = "Resource is always healthy"
        return hs
---
Example for Pod
---
```

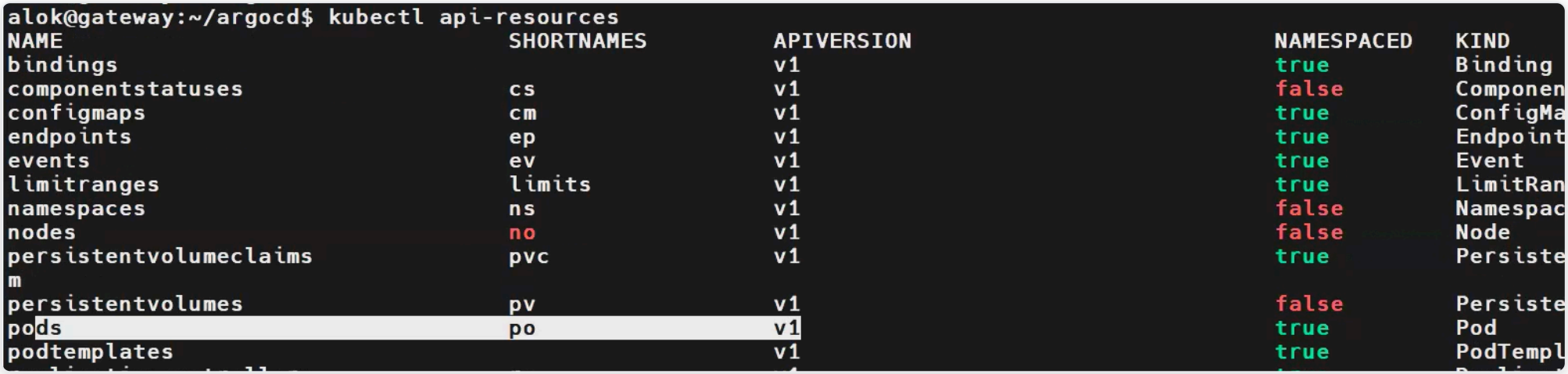
```
kind: ConfigMap
metadata:
  name: argocd-cm
  namespace: argocd
data:
  # Example for a core Pod resource treated as always healthy
  resource.customizations.health.core_Pod: |
    hs = {}
    hs.status = "Healthy"
    hs.message = "Forced to be always healthy"
    return hs
```

```
kind: ConfigMap
metadata:
  name: argocd-cm
  namespace: argocd
data:
  # Example for a core Pod resource treated as always healthy
  resource.customizations.health.core_Pod: |
    hs = {}
    hs.status = "Healthy"
    hs.message = "Forced to be always healthy"
    return hs
```

```
resource.customizations.health.core_Pod: |
```

```
resource.customizations.health.apps_ReplicaSet: |
```


pods po v1 true Pod


alok@gateway:~/argocd$ kubectl api-resources
NAME SHORTNAMES APIVERSION NAMESPACED KIND
bindings v1 true Binding
componentstatuses cs v1 false Componen
configmaps cm v1 true ConfigMa
endpoints ep v1 true Endpoint
events ev v1 true Event
limitranges limits v1 true LimitRan
namespaces ns v1 false Namespac
nodes no v1 false Node
persistentvolumeclaims pvc v1 true Persiste
m
persistentvolumes pv v1 false Persiste
pods po v1 true Pod
podtemplates v1 true PodTempl

```
replicasets                       rs                apps/v1                        true         ReplicaSet
```

```
hs = {}
-- Check if the specific label exists on the ReplicaSet
if obj.metadata.labels ~= nil and obj.metadata.labels["argo-health-override"] == "always-healthy" then
  hs.status = "Healthy"
  hs.message = "Forced to Healthy via application-specific label override"
  return hs
end

-- Fallback to standard ArgoCD ReplicaSet health logic if label is missing
if obj.status ~= nil and obj.status.replicas ~= nil then
  if obj.spec.replicas == obj.status.readyReplicas then
    hs.status = "Healthy"
    hs.message = "ReplicaSet is healthy"
    return hs
  end
end
hs.status = "Progressing"
hs.message = "Waiting for replicas to be ready"
return hs
```

```
Step #2

apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: my-replicaset
  labels:
    argo-health-override: "always-healthy" # This triggers the custom Lua condition
spec:
  replicas: 3
  # ... rest of your spec
```